

Programming Project 1 - Sorting Algorithms

Name: Connor

Course: CS 2430

Semester: Summer 2026

Introduction

Sorting algorithms all solve the same problem but take very different approaches to get there. Some split the data recursively some use heap structures and some just scan back and forth making swaps. Those differences matter because they directly affect how much work the algorithm does especially on larger inputs. This project implements four sorting algorithms in C++ counts comparisons and looks at how they stack up against each other.

Implementation and Methods

Array Generator

Generates a random integer array using C++'s `random_device` as a seed `mt19937` as the engine and `uniform_int_distribution` to keep values between 1 and 100. A new array is generated for each trial.

Merge Sort

Recursively splits the array in half until each piece is one element then merges back in sorted order by comparing the front of each half. Comparisons are counted in the merge step.

Quick Sort

Picks the last element as a pivot and partitions the array so smaller elements go left and larger go right. Recurses on both sides. Comparisons are counted each time an element is checked against the pivot.

Heap Sort

Phase one builds a max heap so the largest element sits at index 0. Phase two repeatedly swaps the root with the last unsorted element shrinks the heap and restores the heap property. Comparisons are counted when a parent is checked against its children.

Shaker Sort

Bubble sort going both directions. Left to right pushes the largest element to the right boundary right to left pushes the smallest to the left boundary. Boundaries shrink each pass. Comparisons counted on every adjacent pair check.

Comparison Counting

Every algorithm increments the same counter whenever two array elements are directly compared. Keeping this rule consistent across all four algorithms is what makes the counts actually comparable to each other.

Verification

Each algorithm was tested by printing the unsorted and sorted arrays side by side across multiple runs. All four consistently produced ascending order output. Size 1 arrays were also tested as an edge case and all algorithms handled them correctly without performing any operations.

Results

10 trials per size arrays randomly generated each run.

10 Runs size = 4

PROCESS MIN MAX AVG

Merge: 4 5 4.80

Quick: 4 6 4.90

Heap: 2 6 3.50

Shaker: 6 6 6.00

10 Runs size = 6

PROCESS MIN MAX AVG

Merge: 8 11 9.50

Quick: 8 14 10.40

Heap: 5 9 7.30

Shaker: 15 15 15.00

10 Runs size = 8

PROCESS MIN MAX AVG

Merge: 14 17 15.80

Quick: 14 27 18.10

Heap: 14 20 16.20

Shaker: 28 28 28.00

Discussion and Analysis

Easiest to implement?

Shaker sort. Having seen bubble sort before made the two direction extension straightforward. No recursion no complex structures just two loops and shrinking boundaries.

Hardest to implement?

Merge sort. Least prior exposure and the recursive splitting plus merging logic had the most room for error. One wrong step and the whole output is wrong which makes it annoying to debug.

How were comparisons counted and why does it matter?

A counter increments every time two elements are directly compared applied the same way in all four

algorithms. If each algorithm counted differently the numbers would not mean anything when put next to each other.

Which was most efficient?

Heap sort and merge sort were closest to each other and both pulled ahead of shaker sort as n grew. This matched expectations since shaker sort is $O(n^2)$ while the others are $O(n \log n)$.

Why dont small sizes perfectly match Big-O?

Big-O describes scaling behavior at large n . At $n = 4, 6$ or 8 the constant factors in each implementation have more influence on the count than the asymptotic complexity does. The patterns become clearer at much larger sizes.

What would stronger evidence look like?

Much larger input sizes and more trials. Worst case testing would also require constructed inputs rather than random ones for example quick sort hits $O(n^2)$ on already sorted arrays which random data rarely produces.

What implementation choices affect performance?

Pivot selection in quick sort memory allocation in merge sort and whether an algorithm can exit early all affect counts. Shaker sort has no early exit so it always produces the same count while quick sort varies a lot based on how the pivot splits the data.

Conclusion

All four algorithms sorted correctly but required different amounts of work to get there. The $O(n \log n)$ algorithms stayed close to each other while shaker sort fell behind noticeably at $n = 8$. Writing these from scratch made the tradeoffs between them much more concrete than reading about them would have.

Summary and Lessons Learned

The biggest takeaway was that you need to fully understand an algorithm before you try to implement it. Any wrong step produces incorrect output and tracing the bug back through recursive calls is tedious. Next time i would work through a few examples by hand before writing any code.

Sources

- Course materials and previous projects
- YouTube videos on merge sort quick sort heap sort and shaker sort
- cppreference.com for C++ standard library documentation